

Database Normalization

Redundancy?

Consider the table:

Sno	Sname	Cname	Duration	fee
1	Anju	BSc	4	600000
2	Binu	BCA	4	550000
3	Anitha	MCA	2	200000
4	Abhinav	BSc	4	600000
5	Ajay	MCA	2	200000
6	Arnold	MCA	2	200000

The Evils of Redundancy

- *Redundancy* is the root of several problems associated with relational schemas:
 - redundant storage, insert/delete/update anomalies

Consider relation obtained from Hourly_Emps:

- Hourly_Emps (ssn, name, lot, rating, hrly_wages, hrs_worked)

Notation: We will denote this relation schema by listing the attributes: SNLRWH

Some FDs on Hourly_Emps:

- *ssn* is the key: $S \rightarrow SNLRWH$
- *rating* determines *hrly_wages*: $R \twoheadrightarrow W$

Example (Contd.)

Problems due to $R \rightarrow W$:

◦ Redundant Storage

◦ Update anomaly: Can we change W in just the 1st tuple of SNLRWH?

◦ Insertion anomaly: What if we want to insert an employee and don't know the hourly wage for his rating?

◦ Deletion anomaly: If we delete all employees with rating 5, we lose the information about the wage for rating 5!

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

*This FD ($R \rightarrow W$) can **NOT** be expressed in terms of the ER model. Only FDs that determine all attributes of a relation (i.e., **key constraints**) can be expressed in the ER model. Therefore we can NOT detect it when we considered Hourly_Emps as an entity set during ER Modeling.*

Functional Dependencies

We say an attribute, B, has a functional dependency on another attribute, A, if for any two records, which have the same value for A, then the values for B in these two records must be the same.

We illustrate this as: $A \rightarrow B$

Example: Suppose we keep track of employee email addresses, and we only track one email address for each employee. Suppose each employee is identified by their unique employee number. We say there is a functional dependency of email address on employee number: employee number $\rightarrow$ email address

- **Definition:** A functional dependency (FD) exists when a set of attributes uniquely determines another attribute. Formally, if $x \rightarrow y$, then knowing the value of x allows us to determine y with certainty.
- **Trivial and Non-Trivial FD:**
 - **Trivial FD:** An FD is trivial if it holds naturally, meaning $x \rightarrow y$ where $y \subseteq x$. Example: $\{a, b \rightarrow a\}$.
 - **Non-Trivial FD:** If $x \rightarrow y$ and $y \not\subseteq x$ it is non-trivial. Example: $a \rightarrow b$, where b is not part of a.

Reasoning About FDs

Given some FDs, we can usually infer additional FDs:

- $ssn \rightarrow did, did \rightarrow lot$ implies $ssn \rightarrow lot$

An FD f is *implied by* a set of FDs F if f holds whenever all FDs in F hold.

- $F^+ = \text{closure of } F$ is the set of all FDs that are implied by F .

Armstrong's Axioms (X, Y, Z are sets of attributes):

- Reflexivity: If $X \subseteq Y$, then $Y \rightarrow X$
- Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z
- Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

These are *sound* and *complete* inference rules for FDs!

Couple of additional rules :

Union: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

Decomposition: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

Closure

- **Closure of FD Set:**

- The closure of a set of functional dependencies is the set of all FDs that can be inferred from the given FDs using Armstrong's axioms: reflexivity, augmentation, and transitivity.
- Computing the closure of a set of FDs can be expensive. (Size of closure is exponential in # attrs!)

Typically, we just want to check if a given FD $X \rightarrow Y$ is in the closure of a set of FDs F . An efficient check:

- Compute attribute closure of X (denoted X^+ wrt F):

```
closure =  $X$ ;  
repeat until there is no change: {  
    if there is an FD  $U \rightarrow V$  in  $F$  such that  $U \subseteq \textit{closure}$ ,  
        then set  $\textit{closure} = \textit{closure} \cup V$ .  
}
```

Problem

Find the attribute closure of the Attribute(s) A,BC,AF for the relation R(A,B,C,D,F) with FDs:

{A→B, BC→D, F→C, D→A}

A ⁺	= {A}	
	= {A U B} = { AB }	A→B
BC ⁺	= {BC}	
	= {BC U D} = {BCD}	BC→D
	= {BCD U A} = { ABCD }	D→A
AF ⁺	= {AF}	
	= {AF U B U C} = {ABCF}	A→B, F→C
	= {ABCF U D} = {ABCDF}	BC→D
	= {ABCDF U A} = { ABCDF }	D→A

Problems-Attribute Closure & Candidate key

Given a relation R with 5 attributes ABCDE and the following FDs: $A \rightarrow B$, $BC \rightarrow E$, and $ED \rightarrow A$.

1. Find the attribute closure of the Attribute A.
2. Is A a candidate key?
3. Find all the candidate keys.
4. Find the prime attributes.
5. Find non-prime attributes.

Prime Attribute: A prime attribute (or key attribute) is an attribute that is part of a candidate key. A candidate key is a minimal set of attributes that can uniquely identify a tuple (row) in a relation (table).

Non-Prime Attribute: A non-prime attribute (or non-key attribute) is an attribute that is not part of any candidate key.

Solution:

$$A^+ = \{AB\}$$

A is not a candidate key since it does not determine all the attributes(i.e A^+ does not contain all the attributes in R)

Candidate keys: Since CD is not in the RHS of any of the FD's, CD is definitely part of the candidate key.

$$CD^+ = \{CD\} \rightarrow \text{not a candidate key}$$

Try for ACD+

$$ACD^+ = \{ABCDE\} \rightarrow \text{ACD is Candidate Key}$$

Other candidate keys(check for A,C,D on the RHS of the FD's and replace it by the LHS, if RHS matches)

$$ACD = \{EDCD\} = \{CDE\}$$

$$CDE = \{CDBC\} = \{BCD\}$$

$$BCD = \{ACD\} \rightarrow \text{already derived}$$

$$\text{Candidate Keys} = \{ACD, CDE, BCD\}$$

$$\text{Prime Attributes} = \{A, B, C, D, E\}$$

$$\text{Non-prime Attributes} = \{ \}$$

Home Assignment

For the given relations and functional dependencies:

- Find all the candidate keys.
 - Find the prime attributes.
 - Find non-prime attributes.
1. $R(ABCDEF)$, $FD=\{AB \rightarrow C, C \rightarrow DE, E \rightarrow F, F \rightarrow A\}$
 2. $R(ABCDEF)$, $FD=\{AB \rightarrow C, C \rightarrow D, C \rightarrow E, E \rightarrow F, F \rightarrow E\}$
 3. $R(ABCDE)$, $FD=\{A \rightarrow D, B \rightarrow A, BC \rightarrow D, AC \rightarrow BE\}$
 4. $R(ABCDE)$, $FD=\{BC \rightarrow D, AC \rightarrow BE, B \rightarrow E\}$
 5. $R(ABCDE)$, $FD=\{B \rightarrow A, A \rightarrow C, BC \rightarrow D, AC \rightarrow BE\}$

Normal Forms

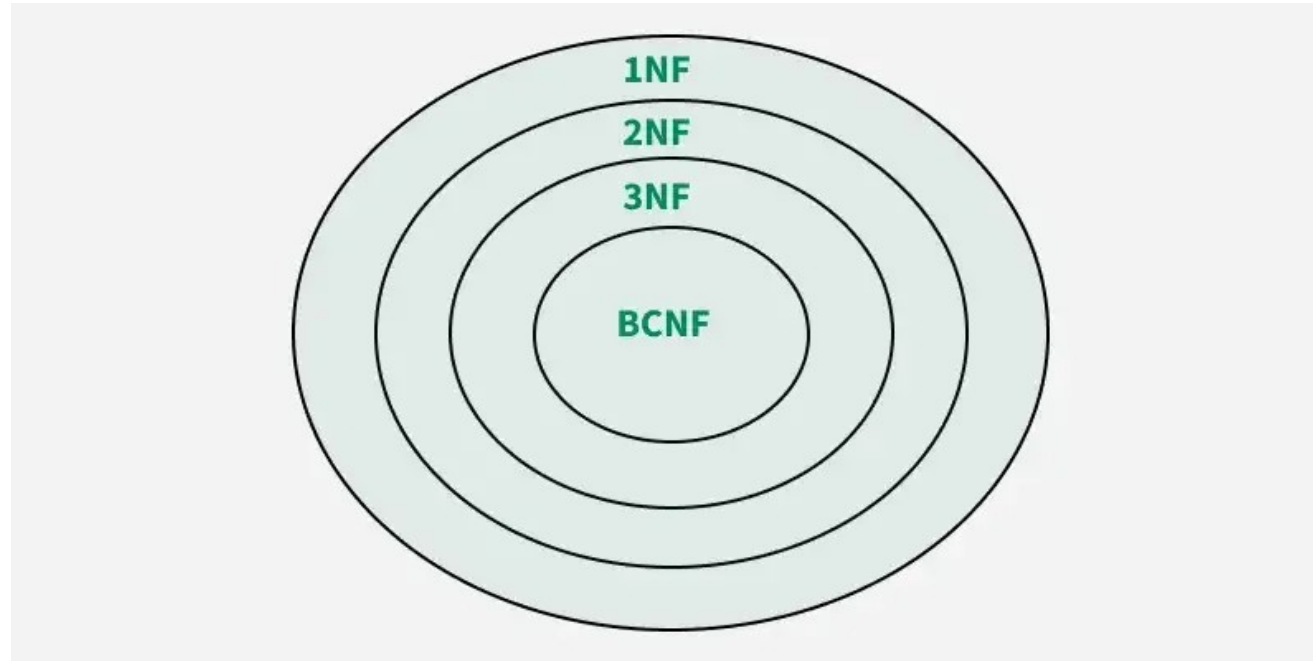
Returning to the issue of schema refinement, the first question to ask is whether any refinement is needed!

If a relation is in a certain *normal form* (1NF, 2NF, BCNF, 3NF etc.), it is known that certain kinds of problems are avoided/minimized. This can be used to help us decide whether decomposing the relation will help.

Role of FDs in detecting redundancy:

- Consider a relation R with 3 attributes, ABC.
 - **No FDs hold:** There is no redundancy here.
 - **Given $A \rightarrow B$:** Several tuples could have the same A value, and if so, they'll all have the same B value!

Levels of Normalization



Each higher level is a subset of the lower level

First Normal Form (1NF)

A table is considered to be in 1NF if all the fields contain only scalar values (as opposed to list of values).

Example (Not 1NF)

ISBN	Title	AuName	AuPhone	PubName	PubPhone	Price
0-321-32132-1	Balloon	Sleepy, Snoopy, Grumpy	321-321-1111, 232-234-1234, 665-235-6532	Small House	714-000-0000	\$34.00
0-55-123456-9	Main Street	Jones, Smith	123-333-3333, 654-223-3455	Small House	714-000-0000	\$22.95
0-123-45678-0	Ulysses	Joyce	666-666-6666	Alpha Press	999-999-9999	\$34.00
1-22-233700-0	Visual Basic	Roman	444-444-4444	Big House	123-456-7890	\$25.00

Author and AuPhone columns are not scalar

1NF - Decomposition

1. Place all items that appear in the repeating group in a new table
2. Designate a primary key for each new table produced.
3. Duplicate in the new table the primary key of the table from which the repeating group was extracted or vice versa.

Example (1NF)

ISBN	Title	PubName	PubPhone	Price
0-321-32132-1	Balloon	Small House	714-000-0000	\$34.00
0-55-123456-9	Main Street	Small House	714-000-0000	\$22.95
0-123-45678-0	Ulysses	Alpha Press	999-999-9999	\$34.00
1-22-233700-0	Visual Basic	Big House	123-456-7890	\$25.00

ISBN	AuName	AuPhone
0-321-32132-1	Sleepy	321-321-1111
0-321-32132-1	Snoopy	232-234-1234
0-321-32132-1	Grumpy	665-235-6532
0-55-123456-9	Jones	123-333-3333
0-55-123456-9	Smith	654-223-3455
0-123-45678-0	Joyce	666-666-6666
1-22-233700-0	Roman	444-444-4444

Second Normal Form (2NF)

For a table to be in 2NF, there are two requirements

- The database is in first normal form
- All **nonkey** attributes in the table must be functionally dependent on the entire primary key / Every non-key attribute is fully dependent on each candidate key of the relation.
- It is not partially dependent on a candidate key.

Note: Remember that we are dealing with non-key attributes

Example 1 (Not 2NF)

Schema -> {Title, PubId, AuId, Price, AuAddress}

1. Key -> {Title, PubId, AuId}
2. {Title, PubId, AuId} -> {Price}
3. {AuId} -> {AuAddress}
4. AuAddress does not belong to a key
5. AuAddress functionally depends on AuId which is a subset of a key

Second Normal Form (2NF)

Example 2 (Not 2NF)

Schema $\rightarrow$ {studio, movie, budget, studio_city}

1. Key $\rightarrow$ {studio, movie}
2. {studio, movie} $\rightarrow$ {budget}
3. {studio} $\rightarrow$ {studio_city}
4. studio_city is not a part of a key
5. studio_city functionally depends on studio which is a proper subset of the key

Example 3 (Not 2NF)

Schema $\rightarrow$ {City, Street, HouseNumber, HouseColor, CityPopulation}

1. key $\rightarrow$ {City, Street, HouseNumber}
2. {City, Street, HouseNumber} $\rightarrow$ {HouseColor}
3. {City} $\rightarrow$ {CityPopulation}
4. CityPopulation does not belong to any key.
5. CityPopulation is functionally dependent on the City which is a proper subset of the key

2NF - Condition for Partial Dependency

Partial Dependency exists if LHS of the FD is a proper subset of Candidate Key **and** RHS is a non-prime attribute.

If a relation has at least one PD, then its not in 2NF

Eg:- R(ABCDEF)

FD={C->F, E->A, EC->D, A->B}

EC⁺->ABCDEF->Candidate Key->No other candidate keys are existing

Prime Attributes->{E,C}

Non-Prime Attributes->{A,B,D,F}

If any one FD is not in 2NF, then the entire relation is not in 2NF. Therefore the above relation is not in 2NF

FD	Partial Dependency	2NF
C->F	Yes	No
E->A	Yes	No
EC->D	No	Yes
A->B	No	Yes

Third Normal Form

Third Normal Form (3NF) ensures that a table is well-structured by eliminating redundant data and avoiding unnecessary dependencies.

A table is in **3NF** if:

1. It is already in **Second Normal Form (2NF)** (i.e., no partial dependencies).
2. **Every non-key attribute depends only on the primary key** and **not on another non-key attribute** (i.e., **no** transitive dependencies).

Explanation:

- Every column should depend **only on the primary key**, not on other non-key columns.
- If one non-key column determines another, that means there is a **transitive dependency**, and we need to **split the table**.

Definition

Reln R with FDs F is in **3NF** if, for all $X \rightarrow A$ in F^+

- $A \rightarrow X$ (called a *trivial* FD), or
- X contains a key for R, or
- A is part of some key for R.

Example

Example (Not in 3NF)

Scheme $\rightarrow$ {Title, PubID, PageCount, Price }

1. Key $\rightarrow$ {Title, PubID}
2. {Title, PubID} $\rightarrow$ {PageCount}
3. {PageCount} $\rightarrow$ {Price}
4. Both Price and PageCount depend on a key hence 2NF
5. Transitively {Title, PubID} $\rightarrow$ {Price} hence not in 3NF

Example

Example 2 (Not in 3NF)

Scheme ->{Studio, StudioCity, CityTemp}

1. Primary Key ->{Studio}
2. {Studio} ->{StudioCity}
3. {StudioCity} ->{CityTemp}
4. {Studio} ->{CityTemp}
5. Both StudioCity and CityTemp depend on the entire key hence 2NF
6. CityTemp transitively depends on Studio hence violates 3NF

Example 3 (Not in 3NF)

Scheme ->{BuildingID, Contractor, Fee}

1. Primary Key ->{BuildingID}
2. {BuildingID} ->{Contractor}
3. {Contractor} ->{Fee}
4. {BuildingID} ->{Fee}
5. Fee transitively depends on the BuildingID
6. Both Contractor and Fee depend on the entire key hence 2NF

BuildingID	Contractor	Fee
100	Randolph	1200
150	Ingersoll	1100
200	Randolph	1200
250	Pitkin	1100
300	Randolph	1200

Condition for 3NF

For a relation to be in 3NF, then for each FD, LHS must be a candidate Key /Super key

OR

RHS is a prime attribute

Example

Roll No	State	Capital
1	Kerala	Thiruvananthapuram
2	Tamil Nadu	Chennai
3	Karnataka	Bangalore
4	Telengana	Hyderabad
5	Kerala	Thiruvananthapuram
6	Telengana	Hyderabad
7	Karnataka	Bangalore
8	Kerala	Thiruvananthapuram

FDs

$R \rightarrow S$

$S \rightarrow C$

Candidate Key

$R^+ = \{RSC\}$

Candidate Key = $\{R\}$

Prime Attributes = $\{R\}$

Non-Prime Attributes = $\{S, C\}$

FD	Partial Dependency	2NF	3NF
$R \rightarrow S$	No	Yes	Yes
$S \rightarrow C$	No	Yes	No (Transitive Dependency)

Problems: Check for 3NF

1. R(ABCD)

FD's={AB→C,C→D}

2. R(ABCD)

FD's={AB→CD,D→A}

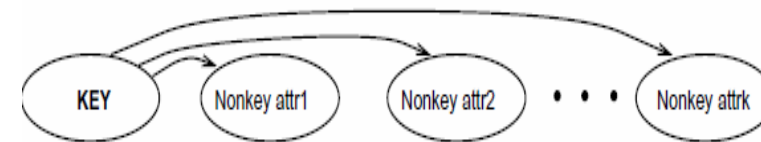
BCNF

BCNF is a stricter form of 3NF that ensures that each determinant in a table is a candidate key. In other words, BCNF ensures that each non-key attribute is dependent only on the candidate key.

The basic condition for any relation to be in BCNF is that it must be in Third Normal Form.

In simpler words, if any column in the table determines another column's value, the determining column must be unique (a candidate key). BCNF fixes any issues left by 3NF

Basic rule for BCNF:



1. Table must be in Third Normal Form.
2. In relation $X \rightarrow Y$, X must be a super-key in a relation. **(LHS of each FD Should be a candidate key/super key)**

Example

Roll No	Name	Adhaar No	Gender
1	Sunu	ABCD1222	F
2	Manu	XYZ123456	M
3	Amith	QWER2344	M
4	Abhijith	JKUL34456	M

FD's

R->N

R->A

A->G

A->R

Candidate Keys

R⁺= {NAGR}

A⁺= {NAGR}

CK= {R,A}

Prime Attributes= {R,A}

Non-Prime Attributes= {N,G}

FD	Partial Dependency	2NF	3NF	BCNF
R->N	No	Yes	Yes	Yes
R->A	No	Yes	Yes	Yes
A->G	No	Yes	Yes	Yes
A->R	No	Yes	Yes	Yes

Example

Example 1 - Address (Not in BCNF)

Scheme $\rightarrow \{City, Street, ZipCode\}$

1. Key1 $\rightarrow \{City, Street\}$
2. Key2 $\rightarrow \{ZipCode, Street\}$
3. No non-key attribute hence 3NF
4. $\{City, Street\} \rightarrow \{ZipCode\}$
5. $\{ZipCode\} \rightarrow \{City\}$
6. Dependency between attributes belonging to a key

Example

Example 2 - Movie (Not in BCNF)

Scheme ->{MovieTitle, MovieID, PersonName, Role, Payment }

1. Key1 ->{MovieTitle, PersonName}
2. Key2 ->{MovieID, PersonName}
3. Both role and payment functionally depend on both candidate keys thus 3NF
4. {MovieID} ->{MovieTitle}
5. Dependency between MovieID & MovieTitle Violates BCNF

Example 3 - Consulting (Not in BCNF)

Scheme ->{Client, Problem, Consultant}

1. Key1 ->{Client, Problem}
2. Key2 ->{Client, Consultant}
3. No non-key attribute hence 3NF
4. {Client, Problem} ->{Consultant}
5. {Client, Consultant} ->{Problem}
6. Dependency between attributes belonging to keys violates BCNF

Problems(Highest Normal Form)

1. $R(ABCDEF)$, $FD=\{AB \rightarrow C, C \rightarrow D, C \rightarrow E, E \rightarrow F, F \rightarrow E\}$
2. $R(ABCDE)$, $FD=\{A \rightarrow D, B \rightarrow A, BC \rightarrow D, AC \rightarrow BE\}$
3. $R(ABCDE)$, $FD=\{BC \rightarrow D, AC \rightarrow BE, B \rightarrow E\}$
4. $R(ABCDE)$, $FD=\{B \rightarrow A, A \rightarrow C, BC \rightarrow D, AC \rightarrow BE\}$

Decomposition

Decomposition of Tables in Database Normalization

Decomposition in database design is the process of breaking a large table into smaller tables while preserving the data and ensuring minimal redundancy. This is usually done to achieve **higher normal forms** (1NF, 2NF, 3NF, BCNF, etc.).

Example

SailorID	SailorName	Address	BoatID	BoatColor	Capacity
1	John	ABC	101	Red	4
2	Alice	DEF	102	Blue	6
1	John	ABC	103	Green	5

◆ **Issues:**

- Repetition of **SailorName** when a sailor owns multiple boats.
- **Boat details** are repeated for every sailor owning the same boat.

To eliminate redundancy, we can **split** the table into **Sailor** and **Boat** tables.

SailorID	SailorName	Address
1	John	ABC
2	Alice	DEF

BoatID	BoatColor	Capacity	SailorID
101	Red	4	1
102	Blue	6	2
103	Green	5	1

◆ **Benefits:**

- No duplication of **SailorName**.
- **Boat details** are stored separately and linked via **SailorID**.

Decomposition of a Relation Scheme

Suppose that relation R contains attributes $A_1 \dots A_n$. A decomposition of R consists of replacing R by two or more relations such that:

- Each new relation scheme contains a subset of the attributes of R (and no attributes that do not appear in R), and
- Every attribute of R appears as an attribute of one of the new relations.

Intuitively, decomposing R means we will store instances of the relation schemes produced by the decomposition, instead of instances of R .

Decompositions should be used only when needed.

- SNLRWH has FDs $S \rightarrow$ SNLRWH and $R \rightarrow W$
- Second FD causes violation of 3NF; W values repeatedly associated with R values. Easiest way to fix this is to create a relation RW to store these associations, and to remove W from the main schema:
 - i.e., we decompose SNLRWH into SNLRH and RW

Types of Decomposition in Database Normalization

1. Lossless (Lossless-Join) Decomposition

- ✓ Ensures that no data is lost when decomposing a table.
- ✓ The original relation can be reconstructed by joining the decomposed tables.

2. Lossy Decomposition

- ✗ Some data may be lost or misinterpreted after decomposition.
- ✗ The original table cannot be fully recovered using joins.

3. Dependency-Preserving Decomposition

- ✓ Ensures that all functional dependencies (FDs) of the original table are maintained after decomposition.
- ✓ Helps maintain consistency and avoids unnecessary joins.

Lossless Join Decompositions

Decomposition of R into X and Y is lossless-join w.r.t. a set of FDs F if, for every instance r that satisfies F:

- $\pi_X(r) \bowtie \pi_Y(r) = r$

It is always true that $r \subseteq \pi_X(r) \bowtie \pi_Y(r)$

Definition extended to decomposition into 3 or more relations in a straightforward way.

It is essential that all decompositions used to deal with redundancy be lossless!

More on Lossless Join

The decomposition of R into X and Y is **lossless-join wrt F** if and only :

- **if** the closure of F contains:

- $X \cup Y \rightarrow X$, or
- $X \cup Y \rightarrow Y$

The Union of Attributes of R1 and R2 must be equal to the attribute of R.
Each attribute of R must be either in R1 or in R2.

$$\text{Att}(R1) \cup \text{Att}(R2) = \text{Att}(R)$$

The intersection of Attributes of R1 and R2 must not be NULL.

$$\text{Att}(R1) \cap \text{Att}(R2) \neq \Phi$$

In particular, the decomposition of R into UV and R - V is lossless-join if U
V holds over R.



A	B	C
1	2	3
4	5	6
7	2	8



A	B
1	2
4	5
7	2

B	C
2	3
5	6
2	8

A	B	C
1	2	3
4	5	6
7	2	8
1	2	8
7	2	3



Decompose to the next Higher Normal form

1. R(ABCDE) and F: {A→C, B→DE, D→C}
2. R(ABCDE) and F: {AB→C, D→E}
3. R(ABCDE) and F: {A→B, B→E, C→D}
4. R(ABCDE) and F: {AB→C, B→D, D→E}
5. R(ABCDE) and F: {BC→D, AC→BE, B→E}
6. R(ABCDE) and F: {B→A, A→C, BC→D, AC→BE}

Dependency Preservation

Dependency Preservation ensures that when a table is decomposed into smaller tables, **all functional dependencies (FDs) of the original table remain preserved** in the decomposed tables. This is important for maintaining **data integrity** and avoiding unnecessary joins to enforce constraints.

Decomposition of R into X and Y is dependency preserving if $(F_X \text{ union } F_Y)^+ = F^+$

1. Dependency Preservation in 3NF

- ✓ 3NF (Third Normal Form) decomposition always preserves dependencies.
- ✓ It removes partial and transitive dependencies while ensuring lossless decomposition.

All dependencies are preserved

Eg:-

R(A,B,C)

F={A→B,B→C,A→C}

2. Dependency Preservation in BCNF

✗ BCNF (Boyce-Codd Normal Form) decomposition does NOT always preserve dependencies.

✓ BCNF ensures a higher level of normalization but may not preserve FDs.

Example Where BCNF Fails to Preserve Dependencies

BCNF prioritizes minimal redundancy, but some FDs may be lost

Eg:-

R(ABCD)

F={AB→CD, D→A}

Example Where BCNF Preserve Dependencies

Eg:-

R(ABCDEH)

FD={A→BC, E→HA}

3NF vs. BCNF in Dependency Preservation

Aspect	3NF	BCNF
Dependency Preservation	Always preserved	Not always preserved
Redundancy	Some redundancy remains	Removes redundancy completely
Data Integrity	Ensures no partial or transitive dependencies	Stronger integrity by removing all anomalies
Join/decomposition	Lossless	Lossy

Comparison of BCNF and 3NF

It is always possible to decompose a relation into a set of relations that are in 3NF such that:

- the decomposition is lossless
- the dependencies are preserved

It is always possible to decompose a relation into a set of relations that are in BCNF such that:

- the decomposition is lossless
- it may not be possible to preserve dependencies.

Fourth Normal Form

Definition

A **relation** is in **Fourth Normal Form (4NF)** if it is in **Boyce-Codd Normal Form (BCNF)** and has **no multi-valued dependencies (MVDs)**.

Multivalued Dependency (MVD):

- Occurs when one attribute in a table determines multiple values of another attribute **independently** of other attributes.

OR

A *multivalued dependency* (MVD) on R , $X \twoheadrightarrow Y$, says that if two tuples of R agree on all the attributes of X , then their components in Y may be swapped, and the result will be two tuples that are also in the relation.

Example: If in a table, a student has multiple phone numbers and multiple mail ids but they(phone & mailid) are **not related to each other**, it creates an MVD.

4NF Rule:

A table is in 4NF **if and only if** it has **no** non-trivial MVDs other than a functional dependency.

Consider the following **EMPLOYEE** table:

Employee_ID	Employee_Name	Mobile_Number	Email_ID
101	Alice	9876543210	alice@email.com
101	Alice	9876543210	alice.work@org.com
101	Alice	7890123456	alice@email.com
101	Alice	7890123456	alice.work@org.com
102	Bob	6789012345	bob@email.com
102	Bob	6789012345	bob.work@org.com

Multi-Valued Dependencies (MVDs) Present

- **Employee_ID** → → **Mobile_Number** (One employee can have multiple mobile numbers).
- **Employee_ID** → → **Email_ID** (One employee can have multiple email addresses).
- However, **Mobile_Number** and **Email_ID** are independent of each other.

Issues with the Current Table

- **Data redundancy:** Employee details are repeated multiple times.
- **Update anomalies:** Changing an employee's name or ID would require multiple updates.
- **Insertion anomalies:** If an employee has only a phone number but no email, we cannot insert it without leaving the email column empty.

Decomposition and 4NF

If $X \twoheadrightarrow Y$ is a 4NF violation for relation R , we can decompose R using the same technique as for BCNF.

1. XY is one of the decomposed relations.
2. All but $X \cup (R-Y)$ is the other.

Fifth Normal Form

A **relation** is in **Fifth Normal Form (5NF)** (also known as **Projection-Join Normal Form, PJNF**) if:

- It is already in **Fourth Normal Form (4NF)**
- It has **no join dependencies** that cause data redundancy

A join dependency (JD) $\bowtie \{R_1, R_2 \dots R_n\}$ is said to hold over a relation R if $R_1 R_2 \dots R_n$ is a lossless-join decomposition of R

Minimal Cover/Canonical Cover of FDs

Irreducible set of FDs

A **minimal cover** (also called **canonical cover**) for a set of Functional Dependencies (**FDs**) is an **equivalent set of FDs** that is:

1. **Minimal in size** (no extra dependencies).
2. **No extraneous attributes** (each attribute in the FD is necessary).
3. **All FDs have a single attribute on the right-hand side** (singleton form).

Steps to Find Minimal Cover

Step 1: Convert to Singleton Right-Hand Side

If any FD has **multiple attributes** on the right-hand side, break it into multiple FDs.

Step 2: Remove Redundant FDs

An **FD is redundant** if removing it does not change the closure of the entire set of FDs.

Step 3: Remove Extraneous Attributes

An **attribute is extraneous** if removing it does not change the closure of the set of FDs.

Given FDs

Let's consider the following set of FDs for a relation $R(A, B, C, D, E)$:

1. $A \rightarrow BC$
2. $B \rightarrow C$
3. $A \rightarrow D$
4. $D \rightarrow E$

Step 1: Convert to Singleton Right-Hand Side

Break $A \rightarrow BC$ into:

- $A \rightarrow B$
- $A \rightarrow C$

Now, the new set is:

1. $A \rightarrow B$
2. $A \rightarrow C$
3. $B \rightarrow C$
4. $A \rightarrow D$
5. $D \rightarrow E$

Step 2: Remove Extraneous Attributes

- In $A \rightarrow C$, we see that $A \rightarrow B$ and $B \rightarrow C$ already exist.
- Since $A \rightarrow B$ and $B \rightarrow C$ imply $A \rightarrow C$, we can **remove** $A \rightarrow C$ as it is redundant.

Now, the updated set is:

1. $A \rightarrow B$
2. $B \rightarrow C$
3. $A \rightarrow D$
4. $D \rightarrow E$

Step 3: Remove Redundant FDs

- **All FDs are necessary**, as removing any FD would change the closure of attributes.

Final Minimal Cover

- $A \rightarrow B$
- $B \rightarrow C$
- $A \rightarrow D$
- $D \rightarrow E$

Canonical Cover

- $A \rightarrow BD$
- $B \rightarrow C$
- $D \rightarrow E$